

An Agentic Intelligent Code Refactoring Assistant for Legacy Systems

R26-SE-008

Project Proposal Report

Amarasinghe W.A.P.M - IT22110848

B.Sc. (Hons) Degree in Information Technology Specialized in
Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

Safe Code Transformation and Validation Agent

R26-SE-008

Project Proposal Report

Amarasinghe W.A.P.M - IT22110848

B.Sc. (Hons) Degree in Information Technology Specialized in
Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

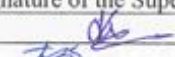
Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Amarasinghe W.A.P.M	IT22110848	

The supervisors should clarify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Supervisor Name	Signature of the Supervisor	Date
Dr. Kalpani Manathunga		17/03/26
Mr. Jeewaka Perera		13-03-2026

ABSTRACT

Legacy software systems are widely used in organizations, but they are difficult to improve because the code is often complex, tightly coupled, and poorly tested [1], [2]. Refactoring can reduce technical debt and improve maintainability, yet it can also introduce unexpected defects [1], [2]. This risk is high in legacy projects where automated tests are missing or incomplete [3], [6].

A major gap in existing refactoring tools is how they validate safety [2], [3]. Most tools rely on syntax correctness, compilation, static analysis, and running existing test suites [2], [3], [9]. When a legacy system has weak or no tests, these approaches cannot reliably confirm that the refactoring preserved the original behavior [3], [6]. As a result, teams must depend on manual review and cautious changes, which slows modernization [3], [10].

This proposal presents a Safe Code Transformation & Validation Agent that applies refactoring changes in a controlled, reversible manner and verifies their safety through multiple validation layers [3], [10]. The agent performs AST-based code transformations, checks syntactic and semantic correctness using static analysis and type checking [9], [10], and validates behavior in two ways: (1) running available project tests [3], and (2) a novel testless validation method [5], [6]. The novelty is Behavioral Fingerprinting with Invariant Mining: the agent runs selected functions with sampled inputs, records behavior signatures (outputs, exceptions, and stability rules such as type and value ranges), and compares results before and after refactoring to detect regressions even without a test suite [5], [6]. If any validation fails, the agent automatically rolls back changes and generates an evidence-based report with confidence scoring [9], [10].

The implementation will use Python, LibCST/AST codemods, and static analysis tools (e.g., mypy and pylint), along with an execution harness for fingerprint collection and comparison [1], [9], [10]. Expected outcomes include safer automated refactoring for legacy code, reduced manual verification effort, and improved confidence in AI-assisted transformations [3], [10]. This can lower maintenance costs and support faster modernization for organizations that depend on legacy systems [1], [2].

ACKNOWLEDGEMENT

I would like to express my sincere appreciation to our research supervisor and co-supervisor for their invaluable guidance, constructive feedback, and academic direction during the conceptualization and proposal development stages of this research. Their expertise, critical insights, and thoughtful recommendations significantly contributed to refining the research problem, defining the system architecture, and clearly structuring the objectives of the Safe Code Transformation & Validation Agent within the broader multi-agent framework.

I am also grateful to the academic staff and panel members of the Software Engineering specialization for their professional guidance and scholarly input during the proposal preparation process. The discussions and feedback received during preliminary evaluations played a vital role in strengthening the theoretical foundation and practical feasibility of the proposed study.

Furthermore, I wish to acknowledge my research team members for their collaborative efforts, intellectual contributions, and active engagement in technical discussions that led to the finalization of the overall project structure. The collective brainstorming sessions and structured planning meetings were instrumental in clearly defining the responsibilities and scope of each component within the system, including the Safe Code Transformation & Validation Agent.

Finally, I extend my sincere gratitude to my family for their continuous encouragement and support throughout the preparation of this research proposal.

TABLE OF CONTENT

Contents

DECLARATION	3
ABSTRACT.....	4
ACKNOWLEDGEMENT	5
TABLE OF CONTENT	6
LIST OF TABLES	7
LIST OF FIGURES	8
LIST OF ABBREVIATIONS.....	9
LIST OF APPENDICES	10
1. INTRODUCTION.....	11
2. OBJECTIVE.....	15
3. METHODOLOGY.....	16
4. HIGH-LEVEL SYSTEM ARCHITECTURE.....	19
5. USER REQUIREMENTS	21
6. COMMERCIALIZATION PLAN.....	24
7. BUDGET AND JUSTIFICATION.....	25
8. WORK BREAKDOWN STRUCTURE (WBS).....	27
9. GANTT CHART.....	29
10. REFERENCES LIST	31
11. APPENDICES.....	32

LIST OF TABLES

Table 1 List of Abbreviations	9
Table 2 Estimated Budget for System Implementation and Experimentation	25
<i>Table 3 Task Distribution Among Team Members</i>	<i>27</i>
<i>Table 4 Project Timeline Alignment</i>	<i>28</i>

LIST OF FIGURES

Figure 1 - Safe code Transformation and Validation Agent - System Overview	19
Figure 2 - Workflow of Safe Code Transformation & Validation Agent	20

LIST OF ABBREVIATIONS

Table 1 List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AST	Abstract Syntax Tree
CFG	Control Flow Graph
IDE	Integrated Development Environment
LLM	Large Language Model
SE	Software Engineering
QA	Quality Assurance
CI/CD	Continuous Integration / Continuous Deployment
API	Application Programming Interface
SDLC	Software Development Life Cycle
OOP	Object-Oriented Programming
SCTVA	Safe Code Transformation & Validation Agent
VCS	Version Control System
NFR	Non-Functional Requirement
SLOC	Source Lines of Code
MTTR	Mean Time To Repair

LIST OF APPENDICES

Appendix A Sample Machine-Executable Refactoring Plan	32
Appendix B High-Level System Architecture Diagram.....	32
Appendix C Component Interaction / Data Flow Diagram.....	32
Appendix D Sample Validation Report / Safety Report	32
Appendix E Behavioral Fingerprinting and Invariant Examples.....	33
Appendix F Preliminary UI Mockups / Dashboard Screens	33
Appendix G - Risk Analysis and Mitigation Table	33
Appendix H Evaluation Metrics and Test Plan	33
Appendix I Sample Legacy Code and Refactored Code Example\.....	34
Appendix J - Ethical Considerations / Data Usage Statement	34

1. INTRODUCTION

1.1 Background and Context

Real-world problem

Legacy software systems remain central to many organizations, but they are difficult to improve safely due to complex dependencies, outdated architecture, limited documentation, and insufficient automated tests [1], [2]. Refactoring is a key practice for reducing technical debt and improving maintainability; however, refactoring can unintentionally change program behavior and introduce regressions—especially in older codebases where hidden side-effects and implicit assumptions are common [1], [2], [3].

Who is affected and why it matters

This problem affects:

- Organizations that depend on long-running systems (finance, government, healthcare, telecom) where failures are costly.
- Developers and maintainers who must manually review changes and cautiously refactor to avoid breaking production.
- End users who suffer from service interruptions, security risks, and slow feature delivery.

Legacy modernization is often delayed because of risk and cost. For example, the U.S. Government Accountability Office reports that agencies typically spend about 80% of IT budgets on operations and maintenance of existing IT, including legacy systems—highlighting the scale of maintenance burden and the need for safe modernization. Industry surveys also indicate that modernizing legacy technology can cost organizations millions annually (e.g., SnapLogic survey findings reported by CIO Dive).

Analysis of existing solutions and limitations

Existing refactoring support typically falls into three categories [2], [4]:

1. IDE refactoring engines (e.g., rename, extract method)
 - o Strength: reliable AST/syntax-aware transformations.
 - o Limitation: mostly ensures structural correctness (syntax/symbol resolution). It does not guarantee behavior preservation without strong tests [2], [3].
 2. Static analysis and quality tools (e.g., code smells, maintainability scoring)
 - o Strength: identifies refactoring opportunities and risks.
 - o Limitation: static rules cannot reliably confirm runtime behavior preservation [3], [9].
 3. Scripted codemods and transformation tools (e.g., codemods, LibCST codemods)
 - o Strength: scalable automated transformations across codebases.
 - o Limitation: usually require human oversight or project test suites to validate safety; transformation correctness is not equal to behavior equivalence [3], [10].
- Overall, most current approaches depend heavily on existing test suites for safety validation [3], [6]. This becomes a major challenge in legacy systems where tests are missing, incomplete, or unreliable [3], [6].

Proposed solution idea and components (function-specific)

This research proposes a Safe Code Transformation & Validation Agent that applies refactorings in a controlled and reversible process and validates safety using multiple layers [3], [10]:

- AST/CST-based transformation (Python) using LibCST codemods for precise and formatting-preserving changes [1], [10].
- Syntactic & semantic validation using parsing, static analysis, and type checking (e.g., pylint, mypy) [9], [10].
- Behavior preservation validation using:
 1. Project tests when available [3], and
 2. Novel Testless Validation using Behavioral Fingerprinting + Invariant Mining, where the agent executes selected functions with sampled inputs, records behavior signatures (outputs/exceptions/properties), and compares pre- vs post-refactoring behavior [5], [6].

Relevant SDG alignment

This function supports:

- SDG 9 (Industry, Innovation and Infrastructure): safer modernization improves reliability and longevity of critical software infrastructure.
- SDG 8 (Decent Work and Economic Growth): reduces developer time wasted on manual verification and production regressions, improving productivity.
- SDG 12 (Responsible Consumption and Production): extending the life of existing systems reduces wasteful rebuilds and repeated redevelopment efforts.

Research cluster alignment

Aligned with common computing research clusters such as:

- Artificial Intelligence in Software Engineering (AI4SE) / Intelligent Software Engineering
- Software Maintenance & Evolution
- Program Analysis and Automated Verification

1.2 Literature Review

Refactoring is traditionally defined as improving internal structure without changing behavior [1], [3]. Ensuring behavior preservation is therefore fundamental, and research has explored formal, static, and dynamic techniques for validating preservation [3], [8]. A systematic mapping study highlights that behavior preservation validation remains challenging and is addressed through approaches such as formal verification, dynamic analysis, and test-based strategies [3].

Research approaches and key findings:

1. Behavior preservation surveys and mapping studies
 - o Summarize that behavior-preserving validation is difficult and often depends on tests or expensive formal methods [3].

2. Refactoring detection and refactoring-aware analysis tools
 - o RefactoringMiner 2.0 is a major refactoring detection approach; research also notes accuracy and threshold limitations in detection tools [4].
3. Formal and consistency checking methods
 - o Model refactoring research proposes formal behavior-preserving consistency checking (e.g., pattern-based checks, Alloy-based methods). These are rigorous but often hard to apply directly to large real-world legacy codebases [8].
4. Refactoring architectures and precondition frameworks
 - o Recent work emphasizes the separation of applicability and behavior-preserving preconditions, reinforcing the need for explicit safety gates before applying refactorings [9], [10].
5. Dynamic analysis and invariant mining
 - o Dynamic invariant detection (e.g., Daikon) demonstrates that invariants can be learned from executions and used to support program understanding and validation [5].
6. Metamorphic testing and oracle problem research
 - o Metamorphic testing is widely used when exact expected outputs are unknown; its principles support testless or low-test validation strategies in practice [6].
7. Recent work on behavior-preserving change characterization
 - o Recent studies quantify how much behavior-preserving changes can be decomposed into known refactoring operations, showing that many behavior-preserving modifications are not fully explained by standard refactoring detectors—indicating gaps in practical behavior-preservation tooling [3], [4].
8. Behavior validation in refactoring contexts
 - o Research prototypes and tools (including mutation-based strategies) show continued effort to validate “behavior maintenance,” but many assume test availability or operate on test code refactoring scenarios [3], [9].

Existing products/services discussed (non-scholarly but relevant):

- LibCST codemods support large-scale Python code transformations while preserving formatting, suitable for reliable automated refactoring pipelines.
- Codemod-style tooling supports scalable refactors but generally expects human oversight and/or tests for validation.

Overall limitation from literature: strong behavior-preservation assurance is often tied to test suites or expensive formal verification, leaving a gap for legacy systems with weak tests [3], [8].

1.3 Research Gap

What is missing

Most refactoring systems can ensure:

- syntactic correctness (AST/CST validity),

- type/compile correctness (where applicable),
- static rule compliance,

but cannot reliably confirm runtime behavior preservation when test suites are missing or weak—a common situation in legacy systems [3]. The behavior-preservation mapping study highlights the importance of behavior validation and the variety of approaches, but also implies that behavior preservation remains difficult and not uniformly solved in practice [3].

Why current approaches are insufficient

- Test-dependent validation fails in test-poor projects: running tests is the dominant practical method, but many legacy systems do not have adequate tests to serve as a reliable safety oracle [3], [6].
- Static checks are not behavior proofs: passing lint/type checks does not guarantee functional equivalence [2], [9].
- Formal methods are often impractical at scale: rigorous model-level proofs can be difficult to apply to real-world legacy codebases with complex runtime dependencies [8], [9].

Novelty and creativity (evidence-based justification)

This proposal introduces Testless Behavior Preservation Validation via:

- Behavioral Fingerprinting: record outputs, exception behavior, and observable runtime signals for selected targets using sampled inputs, pre- and post-refactoring [6].
- Invariant Mining: learn “likely invariants” (type stability, value ranges, non-nullness, exception patterns) from runtime executions, inspired by dynamic invariant detection principles [5].

This novelty directly addresses the legacy-system reality: even without an existing test suite, the agent can produce evidence-based confidence and detect regressions through runtime behavior comparison [5], [6]. Additionally, by combining reversible transformations (rollback) with fingerprint-based validation, the solution moves beyond “structural safety” into behavior-aware refactoring assurance, which is under-supported by existing automated refactoring pipelines [9], [10].

2. OBJECTIVE

2.1 Main Objective

To design and develop a Safe Code Transformation & Validation Agent for legacy software systems that can automatically perform refactoring operations and ensure behavior preservation through safe, reversible, and evidence-based validation techniques [3], [10].

2.2 Specific Objectives

1. To implement AST/CST-based automated refactoring techniques for transforming legacy Python source code in a precise and controlled manner [1], [10].
2. To validate the syntactic correctness and semantic consistency of transformed code using parsing, static analysis, and type checking methods [9], [10].
3. To evaluate refactored code against existing project test suites in order to detect behavioral regressions when tests are available [3].
4. To design and implement a testless behavior validation mechanism using behavioral fingerprinting and invariant mining for test-poor legacy systems [5], [6].
5. To develop a rollback mechanism that automatically restores the original code when a transformation fails validation [9], [10].
6. To produce validation reports and confidence scores that quantify the safety and reliability of each applied refactoring [9], [10].

3. METHODOLOGY

This research adopts a **Design Science Research (DSR)** methodology supported by **experimental evaluation** to design, develop, and assess a **Safe Code Transformation & Validation Agent** for legacy software systems [7]. This approach is appropriate because the study focuses on creating a practical software artifact that solves a real-world problem: safely applying automated refactorings in legacy systems where test coverage is often weak or unavailable [3], [7]. The artifact will be iteratively designed, implemented, demonstrated, and evaluated within the broader multi-agent refactoring framework [7], [10].

The proposed agent receives a **machine-executable refactoring plan** from the upstream planning component and performs a sequence of activities including code transformation, structural validation, behavior validation, safety decision making, rollback if necessary, and safety report generation [9], [10]. The overall goal is to ensure that automated refactoring can be applied in a controlled and trustworthy manner without introducing regressions [3], [10].

3.1 Research Design

The research follows the main phases of Design Science Research. First, the problem is identified as the lack of reliable mechanisms to validate automated refactorings in legacy codebases, especially when test suites are incomplete or missing [3], [7]. Next, the research objective is defined as the development of a safe transformation agent capable of validating whether behavior has been preserved after refactoring [3], [10]. The artifact is then designed and implemented as a prototype system. Finally, the system is demonstrated on selected legacy-style software systems and evaluated using controlled experiments and quantitative performance measures [3], [7], [10].

3.2 System Implementation

The prototype will be implemented in **Python** using **AST/CST-based transformation techniques**, mainly through **LibCST**, to perform syntax-aware and formatting-preserving code modifications [1], [10]. The transformation engine will apply refactoring operations according to the machine-executable refactoring plan produced by the planning agent [1], [10].

The system will contain several main modules. A **transformation engine** will parse source code and apply refactoring actions. A **structural validation module** will check the transformed code through syntax validation, static analysis, and type checking [9], [10]. A **test-based validation module** will run available project tests where possible [3]. A **behavior-preservation module** will compare runtime behavior before and after transformation using **Behavioral Fingerprinting** and **Invariant Mining** [5], [6]. A **safety decision module** will combine all validation evidence and assign a confidence score [9], [10]. Finally, a **rollback manager** will restore the original code if the transformed version is considered unsafe [9], [10].

3.3 Transformation and Validation Process

For each refactoring task, the agent will follow a layered workflow. First, the system receives the machine-executable refactoring plan and identifies the relevant source files, classes, or functions to be modified [10]. The original code is parsed into an AST/CST representation, and a candidate transformed version is generated [1], [10]. Before any change is finalized, the original code is stored so that rollback can be performed if validation fails [9], [10].

After transformation, the system performs **structural validation**. This includes syntax checking to ensure the code can still be parsed correctly, static analysis to detect issues such as unused variables or unreachable code, and type checking to verify type consistency [9], [10]. These checks help ensure that the transformation has not introduced obvious structural or semantic problems.

When test cases are available in the target project, the transformed code is also evaluated through **test-based validation** [3]. Existing test suites are executed on both the original and transformed versions, and the results are compared to identify regressions [3]. However, because many legacy systems do not contain reliable tests, the study introduces an additional testless validation mechanism [3], [6].

3.4 Behavioral Fingerprinting and Invariant Mining

The key novelty of the proposed methodology is the use of **Behavioral Fingerprinting** and **Invariant Mining** to support validation in low-test environments [5], [6]. In Behavioral Fingerprinting, selected functions are executed with sampled inputs before and after refactoring, and their observable runtime behavior is recorded [5], [6]. The fingerprint may include return values, output structure, raised exceptions, and other external behavioral characteristics [5], [6]. By comparing these fingerprints, the system can detect whether the refactoring changed the functional behavior of the code.

In addition to fingerprint comparison, the system will perform **Invariant Mining**. Runtime traces collected during function execution will be used to infer likely behavioral properties such as type consistency, non-null conditions, numeric ranges, size relationships, and exception patterns [5]. These inferred invariants from the original version will be compared with those from the transformed version. If important invariants are violated after refactoring, the transformation will be marked as potentially unsafe [5], [6].

This combined strategy provides an alternative validation mechanism when traditional test oracles are not available, making it particularly suitable for legacy software modernization [3], [5], [6].

3.5 Safety Decision and Rollback

After all validation stages are completed, the system will make a final **safety decision** [9], [10]. The decision will be based on evidence collected from syntax checking, static analysis, type checking, test outcomes, behavioral fingerprint similarity, invariant preservation, and exception consistency [3], [9], [10]. Using these results, the system will classify the transformation as either safe or unsafe and compute a **confidence score** representing the strength of the supporting evidence [9], [10].

If the transformed code fails any mandatory validation stage or shows strong evidence of behavioral regression, the rollback mechanism will automatically restore the original code [9], [10]. If the validation is successful, the transformed code will be accepted and a **safety report** will be generated. This report will include validation outcomes, detected differences, rollback status if applicable, and the final confidence score [9], [10].

3.6 Data Collection

The study will use two main data sources. The first source is **public open-source repositories** that show legacy characteristics such as weak test coverage, complex structure, code smells, or long maintenance history. The second source is **synthetic legacy code samples** created specifically for controlled experiments. These synthetic samples will include common legacy issues such as duplicated logic, long methods, unclear naming, and fragile dependencies.

For each experimental case, the collected data will include the original code, transformed code, refactoring plan, syntax validation results, static-analysis reports, type-checking outputs, test results, behavioral fingerprints, mined invariants, rollback events, confidence scores, and final safety reports. Ethical risk is minimal because the study does not involve human subjects or personal sensitive data; only public or synthetic technical artifacts will be used.

3.8 Ethical and Reliability Considerations

The ethical risk of this research is minimal because it does not involve human participants or personal sensitive data. Only publicly available code repositories and synthetic technical artifacts will be used. To improve reliability, the system will maintain code snapshots, validation logs, and rollback records for each transformation [9], [10]. Runtime validation will be executed in controlled and isolated environments to reduce risk and ensure repeatability [9], [10].

4. HIGH-LEVEL SYSTEM ARCHITECTURE



Figure 1 - Safe code Transformation and Validation Agent - System Overview

4.1 Overview of the Proposed Solution and Its Components

The proposed solution is a Safe Code Transformation & Validation Agent within the broader project, An Agentic Intelligent Code Refactoring Assistant for Legacy Systems [3], [10]. Its main role is to receive a machine-executable refactoring plan, apply the required code changes, validate whether the transformed code is safe, and output refactored code with a safety report [9], [10]. This design follows the proposal's idea of controlled and reversible refactoring using multiple validation layers, including static checks, available tests, and testless behavior validation [3], [5], [6], [9], [10].

The overall high-level flow is:

Machine-Executable Refactoring Plan → Code Transformation → Structural Validation → Safety Decision & Rollback → Refactored Code with Safety Report

In addition to the main flow, the architecture includes supporting validation modules such as test-based validation and testless behavior validation [3], [5], [6]. These are important because legacy systems often have weak or missing test suites, making behavior-preservation checks more difficult [3], [6].



Figure 2 - Workflow of Safe Code Transformation & Validation Agent

4.2 System Diagram of Components

The system diagram presents the working logic of the function from input to output. It begins with the machine-executable refactoring plan, which is produced by the planning part of the overall multi-agent system. The transformation stage then applies AST/CST-based refactoring operations [1], [10]. After that, the transformed code passes through structural validation, where syntax, static analysis, and type checking are performed [9], [10]. Supporting branches such as test-based validation and testless behavior validation provide additional evidence about behavior preservation [3], [5], [6]. Finally, the Safety Decision & Rollback stage decides whether the transformation is safe [9], [10]. If safe, the system outputs refactored code with a safety report; otherwise, rollback restores the original code [9], [10]. This is aligned with the TAF, which states that the agent applies AST-based transformations, validates correctness, supports rollback, assigns confidence scores, and outputs refactored code with safety reports [9], [10].

4.3 Highlight the Individual Component and Its Parts

The function can be divided into several main parts. The input handler receives the refactoring plan. The code transformation module applies the refactoring operations using AST/CST-based techniques such as LibCST codemods [1], [10]. The structural validation module checks syntax, parsing, static analysis, and type correctness [9], [10]. The test-based validation module runs available project tests [3], while the testless behavior validation module compares runtime behavior before and after refactoring using behavioral fingerprinting and invariant mining [5], [6]. A safety decision engine then combines this evidence and determines whether the transformation is safe [9], [10]. If validation fails, the rollback module restores the previous version [9], [10]. If validation succeeds, the output generator produces the refactored code together with the safety report [9], [10].

4.4 Component Interactions

The components interact in a sequential manner [10]. First, the input handler receives the machine-executable refactoring plan and sends it to the transformation module. The transformed code is then checked by the structural validation module [9], [10]. After that, supporting validation modules evaluate the code through tests or testless behavior comparison [3], [5], [6]. Their results are passed to the safety decision engine [9], [10]. If the decision is unsafe, rollback is triggered [9], [10]. If safe, the system produces the final output

as refactored code with safety report [9], [10]. This interaction reflects the proposal’s multi-layer validation approach and the TAF’s transformation-with-validation-and-rollback workflow [9], [10].

4.5 Data Flow Explanation

The data flow begins with the machine-executable refactoring plan as the main input. This plan is used by the transformation module to generate a candidate transformed code version [1], [10]. That version then flows into the structural validation stage and the supporting validation branches [9], [10]. These branches produce validation evidence such as syntax results, static-analysis findings, test outcomes, behavioral fingerprints, and invariant checks [5], [9], [10]. All evidence is then sent to the Safety Decision & Rollback stage [9], [10]. If the candidate is unsafe, rollback restores the original version [9], [10]. If it is safe, the system outputs the final refactored code together with the safety report [9], [10]. This supports the project goal of safe and behavior-preserving refactoring for legacy systems [3], [10].

4.6 Scalability and Security Considerations

The architecture is scalable because its main stages are modular [10]. Transformation, validation, behavior checking, and rollback are separated into different logical parts, making it easier to extend the system with new refactoring rules or validators [9], [10]. The broader project also supports both repository-level and IDE-level operation, which means the architecture can be applied to both large codebases and smaller real-time refactoring tasks. From a security and safety perspective, transformed code should be executed in an isolated environment during validation [9], [10]. This helps prevent harmful side effects. The rollback mechanism is also a major safety feature because it allows the system to restore the original version if validation fails [9], [10]. Additional safety measures include keeping logs, preserving original snapshots, and recording validation evidence for audit purposes [9], [10].

5. USER REQUIREMENTS

5.1 Stakeholder Identification

The proposed Safe Code Transformation & Validation Agent involves several key stakeholders.

Developers and maintainers are the primary users of the system. They need support for applying refactorings safely, especially in legacy systems where manual changes are risky and time-consuming [1], [3]. The proposal highlights that developers currently face high cognitive load and must cautiously review changes to avoid regressions [3], [10].

Software organizations and project teams are also major stakeholders because they depend on legacy systems and need safer modernization with lower maintenance cost and reduced downtime [1], [2]. The proposal states that many organizations in domains such as finance,

government, healthcare, and telecom are affected by legacy-system maintenance challenges [1], [2].

End users of the software systems are indirect stakeholders. They benefit from more reliable software, fewer service interruptions, and safer upgrades [1], [3]. The proposal notes that poor legacy maintenance can lead to service disruptions and slower delivery of improvements [1], [3].

Research supervisors and evaluators are academic stakeholders, since they assess whether the proposed function is feasible, novel, and aligned with the project objectives [7].

Other agents in the multi-agent system are system-level stakeholders because your function depends on the refactoring plan generated upstream and provides validated outputs to the orchestration layer [10]. The TAF shows that the overall system includes analysis, planning, transformation, and orchestration agents working together [10].

5.2 Functional Requirements

1. Refactoring Plan Acceptance
 - The system should accept a machine-executable refactoring plan as input [10].
 - It should use this plan to start the transformation and validation process [10].
2. Automated Code Transformation
 - The system should automatically apply refactoring changes to legacy code [1], [10].
 - It should use AST/CST-based techniques for accurate transformation [1], [10].
3. Syntactic and Semantic Validation
 - The system should check syntax, static analysis, and type correctness after transformation [9], [10].
 - It should prevent invalid or inconsistent code from moving forward [9], [10].
4. Test-Based Validation
 - The system should run available tests after refactoring [3].
 - It should compare test results to detect regressions [3].
5. Testless Behavior Validation
 - The system should validate behavior even when tests are weak or missing [3], [6].
 - It should support safer refactoring in legacy systems with poor test coverage [3], [6].
6. Behavioral Fingerprinting and Invariant Mining
 - The system should compare runtime behavior before and after refactoring [5], [6].
 - It should use outputs, exceptions, and likely invariants to identify behavior changes [5], [6].
7. Safety Decision Making
 - The system should decide whether the transformation is safe or unsafe [9], [10].
 - The decision should be based on all validation evidence [9], [10].
8. Automatic Rollback
 - The system should restore the original code if unsafe changes are detected [9], [10].
 - This ensures safe and reversible refactoring [9], [10].

9. Confidence Score and Safety Report
 - The system should assign a confidence score to the transformation result [9], [10].
 - It should generate a safety report with validation outcomes [9], [10].
10. Refactored Code Output
 - The system should output the validated refactored code with the safety report [9], [10].
 - It should provide the final result only after successful validation [9], [10].

5.3 Non-Functional Requirements

1. Security
 - The system should execute transformed code in a secure and isolated environment [9], [10].
 - It should reduce the risk of harmful side effects during validation [9], [10].
2. Safety
 - The system should prevent unsafe refactorings from being accepted [9], [10].
 - It should allow only validated transformations to become final outputs [9], [10].
3. Reliability
 - The system should provide consistent validation results [3], [9].
 - It should perform rollback correctly whenever unsafe changes are detected [9], [10].
4. Performance
 - The system should complete transformation and validation within a reasonable time [3], [10].
 - It should support practical use in legacy software maintenance tasks [1], [2].
5. Usability
 - The system should provide clear and understandable safety reports [9], [10].
 - It should help developers easily understand validation outcomes [9], [10].
6. Maintainability
 - The system should be easy to update with new refactoring rules and validators [1], [10].
 - It should support future improvements without major redesign [10].
7. Scalability
 - The system should handle both small code units and large legacy repositories [10].
 - It should support both repository-level and IDE-level usage [10].
8. Accuracy
 - The system should accurately identify safe and unsafe refactorings [3], [9].
 - It should minimize false validation decisions [3], [9].
9. Auditability
 - The system should keep logs, validation evidence, and rollback records [9], [10].
 - This should allow later review of transformation decisions [9], [10].

10. Interoperability

- The system should work smoothly with the other agents in the multi-agent framework [10].
- It should support integration with tools such as GitHub and VS Code [10].

6. COMMERCIALIZATION PLAN

In this system has strong commercialization potential within the global software development and software maintenance industry [1], [2]. The primary target market includes software development companies, technology startups, and large enterprises that maintain complex software systems or legacy codebases [1], [2]. These organizations frequently struggle with technical debt, poor code quality, and the increasing complexity of maintaining large software systems over time [1], [2]. Development teams that actively use AI-assisted coding tools such as GitHub Copilot or other large language model-based coding assistants represent a particularly relevant customer segment, as these tools often accelerate code generation without ensuring long-term architectural quality [10]. The main users of the system would include software developers, software architects, DevOps engineers, and quality assurance teams responsible for maintaining and improving code quality [1], [3].

The value proposition of the proposed system lies in its ability to provide an automated and intelligent refactoring workflow that analyzes code quality issues and generates structured refactoring plans before performing code transformations [1], [10]. Unlike traditional static analysis tools that only detect code smells or provide isolated recommendations, the proposed multi-agent framework integrates code analysis, strategic refactoring planning, safe code transformation, and validation into a unified pipeline [3], [9], [10]. The Refactoring Decision and Planning Agent (RDP Agent) plays a key role in this architecture by generating dependency-aware refactoring plans that guide the transformation process and ensure that refactoring actions are applied in an optimal sequence [10].

From a commercialization perspective, the system can be delivered as a developer productivity platform integrated into modern software development environments [10]. The system could be deployed as an integrated development environment (IDE) plugin, a cloud-based code quality analysis service, or an enterprise software maintenance platform integrated with version control systems such as GitHub or GitLab [10]. A Software-as-a-Service (SaaS) revenue model could be adopted, where development teams subscribe to the platform on a monthly or annual basis to analyze repositories, generate refactoring plans, and improve software quality across projects [10].

The operational costs associated with commercial deployment would primarily include cloud infrastructure for code analysis and machine learning processing, platform maintenance and updates, and technical support services for enterprise users. A tiered pricing strategy could be implemented to accommodate different types of customers. For example, smaller development teams and startups could access a basic version of the platform with limited analysis capabilities, while larger enterprises could purchase premium plans that support large-scale repository analysis, advanced refactoring planning, integration with DevOps pipelines, and team collaboration features [10].

The system maintains a competitive advantage by providing a planning-driven automated refactoring solution that considers dependencies, architectural constraints, and expected quality improvements before executing transformations [9], [10]. While many existing tools focus only on detecting code smells or performing individual refactoring actions, the proposed framework introduces an intelligent planning layer that coordinates multiple refactoring operations within a structured workflow [2], [10]. This approach enables safer and more effective large-scale refactoring of complex software systems [3], [10].

In terms of intellectual property, the core innovation of the system lies in the multi-agent architecture and the planning mechanisms used to generate structured refactoring strategies [10]. The algorithms, decision models, and planning architecture developed during this research can be protected through software copyright or intellectual property registration. Additionally, any proprietary machine learning models and training datasets used for predicting refactoring suitability may provide further competitive advantages for a commercial implementation of the system [10].

7. BUDGET AND JUSTIFICATION

The proposed research mainly requires cloud computing resources to run experiments, store datasets, and host the prototype system. Most software tools used in the project such as Python libraries, SonarQube Community Edition, and open-source frameworks are freely available. Therefore, the required budget mainly focuses on minimal cloud infrastructure and hosting services.

Table 2 Estimated Budget for System Implementation and Experimentation

Resource	Purpose	Estimated Cost (LKR)
Cloud Compute Instance (AWS EC2 / DigitalOcean Droplet)	Running code analysis agents and refactoring planning experiments	LKR 7,798.75 / month

Cloud Storage (AWS S3 / Google Cloud Storage)	Storing datasets, repositories, and generated refactoring plans	LKR 935.85 / month
Small Database Service (AWS RDS / Managed PostgreSQL)	Storing software metrics, code quality reports, and experiment results	LKR 3,119.50 / month
Web Hosting / Server Hosting	Hosting the prototype system interface and API	LKR 3,119.50 / month
Domain and Backup Storage	Domain registration and backup of experiment data	LKR 623.90 / month

Total Estimated Budget

Total Cost: 15,597.50 LKR per month

Cloud Compute Instance

A cloud server will be used to run the multi-agent refactoring system, perform static code analysis, and execute experiments on open-source software repositories.

Cloud Storage

Cloud storage services will store datasets, software repositories, experiment results, and generated refactoring plans produced during evaluation.

Database Service

A lightweight database will store structured data such as software metrics, dependency relationships, and refactoring planning outputs.

Web Hosting

A basic hosting service will deploy the prototype system interface, allowing users to submit code quality reports and obtain refactoring plans generated by the planning agent.

Domain and Backup Storage

A small budget is allocated for domain registration and backup storage to ensure reliability and accessibility of the experimental system.

8. WORK BREAKDOWN STRUCTURE (WBS)

The project work is divided among the four team members according to the specialized agents in the multi-agent architecture. Each member is responsible for the design, implementation, and evaluation of a specific agent while collaborating on common tasks such as system integration, testing, and documentation.

Table 3 Task Distribution Among Team Members

Team Member	Component / Agent	Key Responsibilities
Member 1	Code Understanding Agent	Analyzing the source code structure, identifying code smells, generating complexity metrics, and producing structured quality reports.
Member 2	Refactoring Planning Agent	Generating intelligent refactoring strategies based on analysis results, prioritizing refactoring actions, and preparing machine-executable refactoring plans.
Member 3	Safe Code Transformation & Validation Agent	Applying AST-based code transformations, validating syntactic and semantic correctness, performing behavioral validation, and handling rollback when necessary.
Member 4	Developer Interaction & Workflow Orchestration Agent	Managing the interaction between developers and the system, coordinating workflow among agents, presenting refactoring plans and results through a dashboard or IDE interface.

Although each member is responsible for a specific component, several tasks are performed collaboratively to ensure system consistency and integration. These shared tasks include system architecture design, integration of all agents into a unified platform, testing and validation of the complete system, evaluation of experimental results, and preparation of research documentation.

The major development activities of the project are organized into phases that align with the research timeline. The project begins with literature review and system design activities, followed by implementation of the individual agents. After the development stage, the agents are integrated into a unified system and evaluated using selected legacy code repositories. Finally, documentation and final reporting activities are completed.

Table 4 Project Timeline Alignment

Project Phase	Activities	Timeline
Phase 1	Literature review, problem analysis, and research proposal preparation	Month 1 – Month 2
Phase 2	System architecture design and agent specification	Month 3
Phase 3	Implementation of Code Understanding Agent	Month 4 – Month 5
Phase 4	Implementation of Refactoring Planning Agent	Month 4 – Month 5
Phase 5	Implementation of Safe Transformation & Validation Agent	Month 5 – Month 6
Phase 6	Implementation of Developer Interaction & Workflow Orchestration Agent	Month 5 – Month 6
Phase 7	Integration of agents and system testing	Month 7
Phase 8	Experimental evaluation and performance analysis	Month 8
Phase 9	Final documentation, report writing, and presentation preparation	Month 9

The workload distribution among team members is carefully balanced so that each member contributes equally to the development of the proposed system. Each agent requires comparable levels of research, design, and implementation effort. In addition, shared activities such as system integration, testing, evaluation, and documentation are carried out collaboratively by all members. This balanced distribution of tasks ensures efficient progress of the project while allowing each team member to develop expertise in a specific component of the multi-agent architecture.

9. GANTT CHART

The project is organized into multiple phases including literature review, system design, implementation of individual agents, system integration, evaluation, and final documentation. The schedule ensures that each stage of the project is completed within the academic timeline while allowing sufficient time for development, testing, and analysis.

Project Activity	Month 1	Month 2	Month 3	Month 4	Month 5	Month 6	Month 7	Month 8	Month 9
Literature Review	●	●							
Problem Identification	●	●							
System Architecture Design		●	●						
Research Proposal Preparation	●	●							
Development of Code Understanding Agent			●	●					
Development of Refactoring Planning Agent			●	●					
Development of Transformation & Validation Agent				●	●				
Development of Workflow Orchestration Agent				●	●				
System Integration					●	●			
Testing and Validation						●	●		
Experimental Evaluation							●	●	

Final Report Writing							•	•	•
Final Presentation Preparation								•	•

10. REFERENCES LIST

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley Professional, 2018.
- [2] T. Mens and T. Tourwé, “A survey of software refactoring,” *IEEE Transactions on Software Engineering*, vol. 30, no. 2, pp. 126–139, 2004, doi: 10.1109/TSE.2004.1265817.
- [3] E. A. AlOmar, M. W. Mkaouer, C. D. Newman, and A. Ouni, “On preserving the behavior in software refactoring: A systematic mapping study,” *Information and Software Technology*, vol. 140, art. no. 106675, 2021, doi: 10.1016/j.infsof.2021.106675.
- [4] N. Tsantalis, A. Ketkar, and D. Dig, “RefactoringMiner 2.0,” *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 930–950, Mar. 2022, doi: 10.1109/TSE.2020.3007722.
- [5] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, Dec. 2007.
- [6] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, “A survey on metamorphic testing,” *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, Sep. 2016, doi: 10.1109/TSE.2016.2532875.
- [7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.
- [8] G. Rangel, L. Lambers, B. König, H. Ehrig, and P. Baldan, “Behavior preservation in model refactoring using DPO transformations with borrowed contexts,” in *Graph Transformations*, LNCS 5214, pp. 242–256, 2008, doi: 10.1007/978-3-540-87405-8_17.
- [9] J. L. Overbey and R. E. Johnson, “Differential precondition checking: A lightweight, reusable analysis for refactoring tools,” in *Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 303–312, 2011, doi: 10.1109/ASE.2011.6100067.
- [10] B. Sarenac, N. Anquetil, S. Ducasse, and P. Tesone, “A new architecture reconciling refactorings and transformations,” *Journal of Computer Languages*, vol. 80, art. no. 101273, 2024.

11.APPENDICES

Appendix A Sample Machine-Executable Refactoring Plan

Include an example of the structured refactoring plan given as input to your agent.
This can show:

- target file / function
- refactoring type
- transformation steps
- dependency order
- expected constraints

Appendix B High-Level System Architecture Diagram

Include the full architecture diagram of your function within the multi-agent system.
This should show:

- input plan
- code transformation
- structural validation
- test-based validation
- testless validation
- safety decision
- rollback
- output safety report

Appendix C Component Interaction / Data Flow Diagram

Include detailed diagrams such as:

- sequence diagram
- data flow diagram
- module interaction diagram

Appendix D Sample Validation Report / Safety Report

Include a mock example of the final output report generated by your system.
This can include:

- syntax check result
- static analysis result
- test result

- behavioral fingerprint comparison
- invariant preservation result
- confidence score
- rollback decision

Appendix E Behavioral Fingerprinting and Invariant Examples

Include examples of:

- sampled inputs
- outputs before refactoring
- outputs after refactoring
- exceptions
- inferred invariants such as type stability, nullness, value range, return consistency

Appendix F Preliminary UI Mockups / Dashboard Screens

If your system has a user-facing interface, include:

- upload/input screen
- validation status screen
- report view
- rollback notification screen

If there is no UI yet, you can present these as conceptual mockups.

Appendix G - Risk Analysis and Mitigation Table

Include risks such as:

- incorrect transformation
- false safety decision
- weak sampled inputs
- missing dependencies
- unsafe runtime execution
- high validation overhead

And also include mitigation actions.

Appendix H Evaluation Metrics and Test Plan

Include the metrics you mentioned in methodology, such as:

- transformation success rate

- syntax pass rate
- type consistency
- behavioral fingerprint similarity
- invariant preservation rate
- rollback success rate
- regression detection accuracy
- validation overhead

Appendix I Sample Legacy Code and Refactored Code Example

Include a small example showing:

- original code
- selected refactoring
- transformed code
- validation outcome

Appendix J - Ethical Considerations / Data Usage Statement

Since your study does not use human participants, this appendix can briefly state:

- no human subjects involved
- no personal sensitive data collected
- only public or synthetic code artifacts used
- execution performed in controlled environments